

Códigos Python Melhorados

Exercício 1 - Conversor de Tempo

```
def converter_tempo(segundos_totais):
    dias = segundos_totais // 86400
    resto = segundos_totais % 86400

    horas = resto // 3600
    resto %= 3600

    minutos = resto // 60
    segundos = resto % 60

    return dias, horas, minutos, segundos

# Entrada
tempo = int(input("Digite o tempo em segundos: "))

if tempo < 0:
    print("Erro: o valor não pode ser negativo.")
else:
    dias, horas, minutos, segundos = converter_tempo(tempo)

    print("\nResultado:")
    print(f"{dias} dia(s), {horas:02d}:{minutos:02d}:{segundos:02d}")
```

Exercício 2 - Ponte de Wheatstone

```
def calcular_ponte_wheatstone(R1, R2, R3, tensao):
    if R1 == 0:
        raise ValueError("R1 não pode ser zero.")

    Rx = (R2 * R3) / R1

    ramo1 = R1 + Rx
    ramo2 = R2 + R3

    resistencia_total = (ramo1 * ramo2) / (ramo1 + ramo2)

    corrente_total = tensao / resistencia_total
    potencia_total = tensao * corrente_total

    equilibrada = abs((R1 * Rx) - (R2 * R3)) < 1e-9

    return Rx, resistencia_total, corrente_total, potencia_total, equilibrada

try:
    R1 = float(input("Digite R1 (Ω): "))
    R2 = float(input("Digite R2 (Ω): "))
    R3 = float(input("Digite R3 (Ω): "))
    Vfonte = float(input("Digite a tensão da fonte (V): "))

    Rx, R_total, I_total, P_total, equilibrio = calcular_ponte_wheatstone(
        R1, R2, R3, Vfonte
    )

    print("\n--- Resultados ---")
    print(f"Rx = {Rx:.2f} Ω")
    print(f"Resistência Total = {R_total:.2f} Ω")
    print(f"Corrente Total = {I_total:.5f} A")
    print(f"Potência Total = {P_total:.4f} W")

    if equilibrio:
        print("A ponte está equilibrada.")
    else:
        print("A ponte está desequilibrada.")

except ValueError as erro:
    print(f"Erro: {erro}")
```

Exercício 3 - Cálculo Tributário em Cascata

```
def calcular_preco_final(preco_fabrica, aliq_ipi, aliq_icms, aliq_pis_cofins):
    ipi = preco_fabrica * (aliq_ipi / 100)

    base_icms = preco_fabrica + ipi
    icms = base_icms * (aliq_icms / 100)

    base_pis_cofins = base_icms + icms
    pis_cofins = base_pis_cofins * (aliq_pis_cofins / 100)

    preco_final = base_pis_cofins + pis_cofins

    carga_total = (
        (preco_final - preco_fabrica) / preco_fabrica
    ) * 100

    return preco_final, carga_total

preco_fabrica = float(input("Preço de fábrica (R$): "))
aliq_ipi = float(input("IPI (%): "))
aliq_icms = float(input("ICMS (%): "))
aliq_pis_cofins = float(input("PIS/COFINS (%): "))

preco_final, carga_total = calcular_preco_final(
    preco_fabrica,
    aliq_ipi,
    aliq_icms,
    aliq_pis_cofins
)

print("\n--- Resultado ---")
print(f"Preço Final: R$ {preco_final:.2f}")
print(f"Carga Tributária Total: {carga_total:.2f}%")
```

Exercício 4 - Laudo de Sondagem de Solo

```
def classificar_solo(spt, capacidade_carga):
    if spt < 5 or capacidade_carga < 80:
        return "Inapto - reforço necessário"

    elif spt < 10 or capacidade_carga < 150:
        return "Restrito - fundação especial recomendada"

    elif spt >= 30 and capacidade_carga >= 300:
        return "Excelente - solo de alta capacidade"

    else:
        return "Adequado - fundação convencional viável"

spt = int(input("Digite o valor de SPT: "))
capacidade = float(input("Digite a capacidade de carga (kPa): "))

resultado = classificar_solo(spt, capacidade)

print("\nLaudo Técnico:")
print(resultado)
```

Exercício 5 - Controle de Qualidade e Esteira

```
def verificar_solda(resistencia_nominal, resistencia_medida):
    tolerancia = resistencia_nominal * 0.08
    desvio = resistencia_medida - resistencia_nominal

    if abs(desvio) <= tolerancia:
        return "Solda aprovada"

    percentual = (desvio / resistencia_nominal) * 100
```

```

return (
    f"Solda reprovada. "
    f"Desvio: {desvio:.2f} MPa ({percentual:.2f}%" )

def verificar_esteira(velocidade_atual, velocidade_alvo):
    limite_superior = velocidade_alvo * 1.10
    limite_inferior = velocidade_alvo * 0.90

    if velocidade_atual > limite_superior:
        return "Velocidade alta: risco de posicionamento incorreto"

    elif velocidade_atual < limite_inferior:
        return "Velocidade baixa: gargalo na produção"

    else:
        return "Velocidade OK"

# Parte A - Solda
resistencia_nominal = float(input("Resistência nominal: "))
resistencia_medida = float(input("Resistência medida: "))

print("\nResultado da Solda:")
print(verificar_solda(resistencia_nominal, resistencia_medida))

# Parte B - Esteira
velocidade_atual = float(input("\nVelocidade atual: "))
velocidade_alvo = float(input("Velocidade alvo: "))

print("\nResultado da Esteira:")
print(verificar_esteira(velocidade_atual, velocidade_alvo))

```